

Sistema de Inteligência Artificial AquaSense

Rede Neural Multi-Output para Optimização Automática de Aquários

Autores: Rui Outeiro, Emanuel Carvalho, Paulo Jadaugy

Data: Fevereiro 2026

Versão: 1.0.0

Índice

- Sistema de Inteligência Artificial AquaSense
 - Introdução
 - Fundamentos Teóricos de Redes Neurais
 - Arquitectura do Modelo PhotoperiodNet
 - Dados e Processo de Treino
 - Optimização e Treino
 - Métricas de Avaliação
 - Resultados Experimentais
 - Conclusões

Introdução

O módulo de IA do AquaSense foi desenvolvido para **optimizar automaticamente as condições do aquário** através da análise de dados de sensores em tempo real. O sistema recebe três entradas (turbidez, pH, temperatura) e produz três saídas accionáveis (ajuste de fotoperíodo, TPA recomendada, ajuste de alimentação).

Problema

Utilizadores sem experiência enfrentam dificuldades em correlacionar múltiplos parâmetros interdependentes. A turbidez elevada pode indicar excesso de luz (algas), mas também excesso de alimentação ou necessidade de TPA. A IA resolve este problema analisando todos os parâmetros simultaneamente.

Solução Proposta

Uma **rede neural multi-output** treinada com dados sintéticos baseados em regras de aquariofilia. O modelo aprende relações não-lineares entre os parâmetros e generaliza para cenários não vistos durante o treino.

Entrada	Intervalo	Descrição
Turbidez	0-100%	Claridade da água (0 = cristalina)
pH	6.0-8.5	Acidez/alcalinidade
Temperatura	20-31°C	Temperatura da água

Saída	Intervalo	Descrição
-------	-----------	-----------

Saída	Intervalo	Descrição
Ajuste Fotoperíodo	-12 a 0h	Redução de horas de luz
TPA	0-100%	Troca parcial de água recomendada
Alimentação	0-100%	Percentagem da alimentação normal

Fundamentos Teóricos de Redes Neurais

2.1 Neurónio Artificial

O **neurónio artificial** (perceptrão) é a unidade básica das redes neurais. Inspirado no neurónio biológico, realiza uma soma ponderada das entradas, adiciona um bias e aplica uma função de activação.

[!NOTE]

Equação do Neurónio

$$y = f(\sum w_i x_i + b) = f(w^T x + b)$$

Onde: y = saída, x_i = entradas, w_i = pesos, b = bias, f = função de activação

2.2 Camadas Lineares (Fully Connected)

Uma **camada linear** (ou *dense layer*) é uma transformação afim que mapeia um vector de entrada para um vector de saída através de uma matriz de pesos W e um vector de bias b .

[!NOTE]

Transformação Linear

$$z = Wx + b$$

Onde: $z \in \mathbb{R}^m$ = saída, $W \in \mathbb{R}^{m \times n}$ = matriz de pesos, $x \in \mathbb{R}^n$ = entrada, $b \in \mathbb{R}^m$ = bias

Parâmetros treináveis: $m \times n + m = m(n + 1)$

Exemplo: A primeira camada do PhotoperiodNet é **Linear(3, 32)**, com $3 \times 32 + 32 = 128$ parâmetros.

2.3 Funções de Activação

As funções de activação introduzem **não-linearidade**, permitindo que a rede aprenda relações complexas. Sem elas, múltiplas camadas lineares seriam equivalentes a uma única transformação linear.

2.3.1 ReLU (Rectified Linear Unit)

A função **ReLU** é a activação mais usada em camadas ocultas devido à sua simplicidade e eficácia.

[!NOTE]

FUNÇÃO RELU

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x, & \text{se } x > 0 \\ 0, & \text{se } x \leq 0 \end{cases}$$

Domínio: $\mathbb{R}$ | **Imagem:** $[0, +\infty)$

Derivada: $f'(x) = 1$ se $x > 0$, $f'(x) = 0$ se $x \leq 0$

[!NOTE]

DERIVADA DA RELU

$$\frac{d}{dx} \text{ReLU}(x) = \begin{cases} 1, & \text{se } x > 0 \\ 0, & \text{se } x \leq 0 \end{cases}$$

A derivada simples (0\$ ou 1\$) evita o *vanishing gradient problem* que afecta sigmoid/tanh em redes profundas.

Vantagens da ReLU:

- **Esparsidade:** Neurónios negativos = 0, criando representações esparsas eficientes
- **Gradiente não saturante:** Para $x > 0$, gradiente = 1 (treino mais rápido)
- **Computacionalmente eficiente:** Apenas uma comparação (vs. exponenciais na sigmoid)
- **Convergência 6x mais rápida** que tanh (Krizhevsky et al., 2012)

Desvantagens:

- **Dying ReLU:** Neurónios com entrada sempre negativa "morrem" (gradiente = 0 permanente)
- **Não centrada em zero:** Saídas ≥ 0 podem causar zig-zagging na optimização

2.3.2 Função Sigmoid

A **sigmoid** mapeia valores para (0, 1), ideal para probabilidades ou valores normalizados.

[!NOTE]

FUNÇÃO SIGMOID (LOGÍSTICA)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Domínio: $\mathbb{R}$ | **Imagem:** $(0, 1)$

$$\sigma(0) = 0.5 \mid \lim_{x \rightarrow +\infty} \sigma(x) = 1 \mid \lim_{x \rightarrow -\infty} \sigma(x) = 0$$

DERIVADA DA SIGMOID

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

Derivada máxima $= 0.25$ (quando $x = 0$). Este valor pequeno causa *vanishing gradient* em redes profundas, por isso usamos sigmoid apenas nas saídas.

Uso no PhotoperiodNet:

- **Ajuste:** $-\text{sigmoid}(x) \rightarrow [-1, 0] \times 12 = [-12\text{h}, 0\text{h}]$
- **TPA:** $\text{sigmoid}(x) \rightarrow [0, 1] \times 100 = [0\%, 100\%]$
- **Alimentação:** $\text{sigmoid}(x) \rightarrow [0, 1] \times 100 = [0\%, 100\%]$

2.3.3 Comparação de Funções de Activação

Função	Fórmula	Intervalo	Derivada	Uso
ReLU	$\max(0, x)$	$[0, +\infty)$	0 ou 1	Camadas ocultas
Sigmoid	$1/(1+e^{-x})$	$(0, 1)$	$\sigma(1-\sigma)$	Saídas normalizadas
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	$(-1, 1)$	$1 - \tanh^2$	RNNs
Leaky ReLU	$\max(\alpha x, x)$	$\mathbb{R}$	α ou 1	Evitar dying ReLU
Softmax	$e^i / \sum e^j$	$(0,1), \Sigma=1$	complexa	Classificação

2.4 Backpropagation e Gradiente Descendente

O **backpropagation** calcula gradientes da loss em relação aos pesos, propagando o erro da saída para as camadas anteriores usando a regra da cadeia.

[!NOTE]

REGRA DA CADEIA

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w}$$

O gradiente de L em relação a w é o produto dos gradientes ao longo do caminho computacional.

[!NOTE]

ACTUALIZAÇÃO DE PESOS (GRADIENT DESCENT)

$$w_{t+1} = w_t - \eta \cdot \frac{\partial L}{\partial w_t}$$

η (**eta**): *learning rate* (taxa de aprendizagem)

Se gradiente > 0 , diminuimos w . Se gradiente < 0 , aumentamos w .

Variantes do Gradient Descent

Variante	Batch	Características
Batch GD	Todo o dataset	Estável mas lento
SGD	1 amostra	Rápido mas ruidoso
Mini-batch	n amostras (32)	Equilíbrio - usado no PhotoperiodNet

Arquitectura do Modelo PhotoperiodNet

```
flowchart TD
    %% Node Definitions
    INP["INPUT (3 features)"]
    INP["Turbidez, pH, Temperatura"] - normalizados
    FC1["FC1: Linear(3→32) + ReLU + Dropout(10%)"]
```

```
FC2["FC2: Linear(32→32) + ReLU + Dropout(10%)"]
SHARED["SHARED: Linear(32→16) + ReLU"]

HEAD1["HEAD 1
Ajuste
-Sigmoid
[-1, 0]"]
HEAD2["HEAD 2
TPA
Sigmoid
[0, 1]"]
HEAD3["HEAD 3
Alimentação
Sigmoid
[0, 1]"]

%% Styling
style INP fill:#1a2c23,stroke:#52b788,stroke-width:2px,color:#fff
style FC1 fill:#2c1e14,stroke:#fca311,stroke-width:2px,color:#fff
style FC2 fill:#2c1e14,stroke:#fca311,stroke-width:2px,color:#fff
style SHARED fill:#2c1e14,stroke:#fca311,stroke-width:2px,color:#fff

style HEAD1 fill:#1a1a2e,stroke:#7209b7,stroke-width:2px,color:#fff
style HEAD2 fill:#1a1a2e,stroke:#7209b7,stroke-width:2px,color:#fff
style HEAD3 fill:#1a1a2e,stroke:#7209b7,stroke-width:2px,color:#fff

%% Connections
INP --> FC1
FC1 --> FC2
FC2 --> SHARED
SHARED --> HEAD1
SHARED --> HEAD2
SHARED --> HEAD3
```

Detalhes das Camadas

Camada	Tipo	Entrada→Saída	Activação	Parâmetros
fc1	Linear	3→32	ReLU	128
dropout1	Dropout(0.1)	32→32	-	0
fc2	Linear	32→32	ReLU	1,056
dropout2	Dropout(0.1)	32→32	-	0
shared	Linear	32→16	ReLU	528
head_adj	Linear	16→1	-Sigmoid	17
head_tpa	Linear	16→1	Sigmoid	17
head_feed	Linear	16→1	Sigmoid	17

Camada	Tipo	Entrada→Saída	Activação	Parâmetros
Total				1,763

Dropout como Regularização

[!NOTE]

DROPOUT

$$h_{\text{drop}} = \frac{m \cdot h}{1 - p}$$

m: máscara binária (Bernoulli) | **p** = 0.1: probabilidade de desactivar

Durante treino: 10% dos neurónios são zerados aleatoriamente

Durante inferência: todos os neurónios activos

Multi-Head Output

As três "cabeças" partilham a representação aprendida pelas camadas ocultas, mas especializam-se cada uma na sua tarefa. Isto é mais eficiente que três modelos separados e permite *transfer learning* implícito entre tarefas relacionadas.

Dados e Processo de Treino

4.1 Geração de Dados Sintéticos

O modelo foi treinado com **10.000 amostras sintéticas** geradas por regras especializadas de aquariofilia.

Distribuições das Features

[!NOTE]

TURBIDEZ: BETA(2, 5) × 100

$$f(x; \alpha=2, \beta=5) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha, \beta)}$$

Média: $2/7 \approx 28.6\%$ | **Moda:** 20%

Maioria das amostras em valores baixos (água limpa) - realista para aquários bem mantidos.

[!NOTE]

PH: NORMAL(7.0, 0.4), TRUNCADA [6.0, 8.5]

$$f(x; \mu=7, \sigma=0.4) = \frac{1}{\sigma\sqrt{2\pi}} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Distribuição centrada no pH neutro (7.0), típico de água de aquário.

[!NOTE]

TEMPERATURA: NORMAL(25.5, 2.0), TRUNCADA [20, 31]

$$f(x; \mu=25.5, \sigma=2) \text{ - temperatura típica de aquário tropical}$$

Média: 25.5°C | **Desvio Padrão:** 2.0

4.2 Regras Base (Ground Truth)

Os labels são gerados por regras especializadas. A turbidez é o driver principal:

Turbidez	Nível	Ajuste Luz	TPA	Alimentação
0-20%	Normal	0h	15%	100%
20-40%	Baixo	-1 a -3h	20-30%	100%
40-60%	Moderado	-3 a -5h	30-50%	75%
60-80%	Alto	-5 a -8h	50-70%	50%
80-100%	Crítico	-8 a -10h	70-80%	0%

pH e Temperatura actuam como multiplicadores de risco quando fora dos intervalos ideais (pH 6.8-7.2, Temp 22-28°C).

4.3 Normalização (StandardScaler)

[!NOTE]

Z-SCORE NORMALIZATION

$$x_{norm} = \frac{x - \mu}{\sigma}$$

μ: média do conjunto de treino | **σ:** desvio padrão **Importante:** O scaler é ajustado apenas nos dados de treino e guardado em `models/scaler.pkl`

4.4 Divisão Train/Test

- **80% Treino:** 8,000 amostras
- **20% Teste:** 2,000 amostras (nunca vistas durante treino)
- **Seed fixa:** 42 (reprodutibilidade)

Optimização e Treino

5.1 Função de Perda: MSE

[!NOTE]

MEAN SQUARED ERROR (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Média dos quadrados das diferenças entre valores reais (y) e previsões ($\hat{y}$).

Penaliza mais erros grandes devido ao quadrado.

5.2 Optimizador Adam

Adam (Adaptive Moment Estimation) é um algoritmo de optimização estocástica proposto por Kingma & Ba (2014). Combina as vantagens do **SGD com Momentum** (que acelera a convergência em direcções consistentes) e do **RMSprop** (que adapta a learning rate por parâmetro).

[!NOTE]

Definição Formal: Adam

Adam mantém estimativas de médias móveis exponenciais do primeiro momento (média) e do segundo momento (variância não centrada) dos gradientes, usando estas estimativas para adaptar a taxa de aprendizagem de cada parâmetro individualmente.

5.2.1 Motivação e Contexto Histórico

Antes do Adam, os optimizadores tinham limitações específicas:

- **SGD:** Learning rate fixa para todos os parâmetros; convergência lenta em superfícies com curvatura variável
- **SGD + Momentum:** Acelera convergência mas não adapta por parâmetro
- **AdaGrad:** Adapta LR por parâmetro mas acumula gradientes indefinidamente, causando $LR \rightarrow 0$
- **RMSprop:** Resolve AdaGrad com média móvel, mas sem momentum

O **Adam** unifica estas técnicas, sendo robusto a hiperparâmetros e eficiente em memória.

5.2.2 Algoritmo Completo

[!NOTE]

ALGORITMO: ADAM (ADAPTIVE MOMENT ESTIMATION)

1. **Entrada:** α (*learning rate*), β_1 , β_2 (taxas de decaimento), ϵ (estabilidade), θ_0 (parâmetros iniciais)
2. **Inicializar:** $m_0 = 0$ (1° momento), $v_0 = 0$ (2° momento), $t = 0$
3. **Repetir até convergência:**
 - $t \leftarrow t + 1$
 - $g_t \leftarrow \nabla_{\theta} L(\theta_{t-1})$ (calcular gradiente)
 - $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (actualizar 1° momento)
 - $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (actualizar 2° momento)
 - $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (correção de bias 1° momento)
 - $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (correção de bias 2° momento)
 - $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (actualizar parâmetros)
4. **Retornar:** θ_t (parâmetros optimizados)

5.2.3 Fórmulas Matemáticas Detalhadas

[!NOTE]

PASSO 1: CÁLCULO DO GRADIENTE

$$g_t = \nabla_{\theta} L(\theta_{t-1}) = \partial L / \partial \theta$$

O gradiente g_t é o vector de derivadas parciais da função de perda L .

PASSO 2: ACTUALIZAÇÃO DO PRIMEIRO MOMENTO (MÉDIA)

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

$\beta_1 = 0.9$: Suaviza oscilações e acelera em direcções consistentes (*momentum*).

PASSO 3: ACTUALIZAÇÃO DO SEGUNDO MOMENTO (VARIÂNCIA)

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

$\beta_2 = 0.999$: Mede a "magnitude histórica" dos gradientes para cada parâmetro.

PASSO 4: CORRECÇÃO DE BIAS (BIAS CORRECTION)

$$\hat{m}_t = m_t / (1 - \beta_1^t) \quad \text{e} \quad \hat{v}_t = v_t / (1 - \beta_2^t)$$

Compensa o facto de m_0 e v_0 serem inicializados a zero.

PASSO 5: ACTUALIZAÇÃO DOS PARÂMETROS

$$\theta_t = \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

$\alpha = 0.001$: Magnitude base do passo no PhotoperiodNet. $\epsilon = 10^{-8}$: Evita a divisão por zero.

5.2.4 Hiperparâmetros do Adam

Parâmetro	Valor Típico	PhotoperiodNet	Função
α (learning rate)	0.001	0.001	Magnitude do passo de actualização
β_1	0.9	0.9	Decaimento do 1º momento (momentum)
β_2	0.999	0.999	Decaimento do 2º momento (adaptativo)
ϵ	10^{-8}	10^{-8}	Estabilidade numérica

5.2.5 Comparação com Outros Optimizadores

Optimizador	Fórmula de Actualização	Vantagens	Desvantagens
SGD	$\theta \leftarrow \theta - \alpha \cdot g$	Simples, garantias teóricas	Lento, sensível a LR

Optimizador	Fórmula de Atualização	Vantagens	Desvantagens
SGD + Momentum	$v \leftarrow \gamma v + \alpha \cdot g$ $\theta \leftarrow \theta - v$	Acelera convergência	Mais hiperparâmetros
AdaGrad	$\theta \leftarrow \theta - \alpha \cdot g / \sqrt{(\sum g^2)}$	Adapta LR por parâmetro	LR→0 com o tempo
RMSprop	$\theta \leftarrow \theta - \alpha \cdot g / \sqrt{v}$	Resolve problema AdaGrad	Sem momentum
Adam	$\theta \leftarrow \theta - \alpha \cdot \hat{m} / \sqrt{\hat{v}}$	Momentum + Adaptativo + Bias correction	Pode não generalizar bem em alguns casos

5.2.6 Porque Usamos Adam no PhotoperiodNet

[!TIP]

Justificação da escolha:

- **Convergência rápida:** Modelo pequeno (1,763 parâmetros) beneficia de optimização eficiente
- **Robusto a hiperparâmetros:** Valores padrão funcionam bem sem tuning extensivo
- **Multi-output:** Diferentes cabeças podem ter gradientes de magnitudes diferentes; Adam adapta automaticamente
- **Mini-batch:** Adam funciona bem com batch size pequeno (32) devido ao momentum
- **Standard da indústria:** Optimizador mais usado em deep learning moderno

5.2.7 Implementação em PyTorch

```
# Configuração do Adam no PhotoperiodNet
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001,           # α: learning rate
    betas=(0.9, 0.999), # (β1, β2): taxas de decaimento
    eps=1e-8,          # ε: estabilidade numérica
    weight_decay=0     # L2 regularization (não usado)
)

# Loop de treino simplificado
for epoch in range(max_epochs):
    for batch_X, batch_y in dataloader:
        optimizer.zero_grad()      # Limpar gradientes anteriores
        predictions = model(batch_X) # Forward pass
        loss = criterion(predictions, batch_y) # Calcular loss
        loss.backward()             # Backpropagation (calcula gradientes)
        optimizer.step()            # Adam actualiza parâmetros
```

5.2.8 Variantes do Adam

Existem várias extensões do Adam para casos específicos:

Variante	Modificação	Caso de Uso
AdamW	Weight decay desacoplado	Melhor regularização L2
AMSGrad	Máximo histórico de vt	Convergência garantida
RAdam	Warmup automático da variância	Início de treino mais estável
NAdam	Nesterov momentum	Convergência ligeiramente mais rápida

5.3 Learning Rate Scheduling

[!NOTE]

REDUCELRONPLATEAU

$$\eta_{\text{new}} = \eta_{\text{old}} \times \text{factor} \quad \text{(se loss não melhora em } \text{patience} \text{ épocas)}$$

factor = 0.5: reduz a taxa de aprendizagem (η) para metade

patience = 20: espera 20 épocas sem melhoria antes de reduzir

5.4 Early Stopping

O treino pára quando a **validation loss** não melhora durante 50 épocas consecutivas, prevenindo overfitting.

Algoritmo: Early Stopping

```

1. Inicializar best_loss = ∞, patience_counter = 0
2. Para cada época:
    * Se val_loss < best_loss: best_loss=val_loss, patience_counter=0,
    guardar modelo
    * Senão: patience_counter += 1
    * Se patience_counter ≥ 50: PARAR

```

5.5 Validação Cruzada K-Fold

[!NOTE]

K-FOLD CROSS VALIDATION (K=5)

$$\text{Performance} = \frac{1}{K} \sum \text{metric}_{\text{fold}_i}$$

O dataset é dividido em 5 partes. Cada fold usa 4 partes para treino e 1 para validação. Garante que o modelo generaliza bem independentemente da divisão dos dados.

Hiperparâmetros do Treino

Parâmetro	Valor	Justificação
Batch Size	32	Equilíbrio memória/estabilidade

Parâmetro	Valor	Justificação
Max Epochs	500	Limite máximo (early stopping activo)
Learning Rate	0.001	Valor padrão para Adam
Patience	50	Épocas sem melhoria antes de parar
Dropout	0.1	Regularização leve para rede pequena

Métricas de Avaliação

6.1 MAE (Mean Absolute Error)

[!NOTE]

MEAN ABSOLUTE ERROR (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Média dos erros absolutos. Interpretável nas unidades originais (horas, %).

Vantagem: Robusto a *outliers* (não penaliza erros grandes excessivamente).

Interpretável nas unidades originais (horas, %).

6.2 MSE (Mean Squared Error)

[!NOTE]

MEAN SQUARED ERROR (MSE)

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

Média dos erros ao quadrado. Penaliza mais erros grandes.

Usada como função de perda durante o treino.

6.3 RMSE (Root Mean Squared Error)

[!NOTE]

ROOT MEAN SQUARED ERROR (RMSE)

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{(1/n) \sum (y_i - \hat{y}_i)^2}$$

Raiz quadrada do MSE. Mesma unidade que os dados originais.

Mais sensível a erros grandes que MAE.

6.4 R² (Coeficiente de Determinação)

[!NOTE]

R² SCORE

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

Proporção da variância explicada pelo modelo. **$R^2 = 1$** : previsão perfeita | **$R^2 = 0$** : modelo $\approx$ média

$R^2 < 0$: modelo pior que média (problema sério)

6.5 Accuracy por Threshold

[!NOTE]

ACCURACY@THRESHOLD

$$\text{Acc}_t = \frac{1}{n} \sum 1[|y_i - \hat{y}_i| < t] \times 100\%$$

Percentagem de previsões com erro absoluto menor que threshold t.

Exemplo: Accuracy@1h = % previsões com erro < 1 hora

Resumo das Métricas

Métrica	Fórmula	Intervalo	Objectivo
MAE	$(1/n) \sum$	$y - \hat{y}$	\$
MSE	$(1/n) \sum (y - \hat{y})^2$	$[0, +\infty)$	Minimizar
RMSE	$\sqrt{\text{MSE}}$	$[0, +\infty)$	Minimizar
R ²	$1 - SS_{\text{res}}/SS_{\text{tot}}$	$(-\infty, 1]$	Maximizar ($\rightarrow 1$)
Acc@t	% erros < t	$[0, 100]\%$	Maximizar

Resultados Experimentais

7.1 Distribuição dos dados

 Distribuição dos dados executando através do notebook:

7.2 Métricas Finais

 Treino feito através do notebook:

Resultados do Modelo

MAE Fotoperíodo	Accuracy <1h	MAE TPA	Acc TPA <10%
0.20h	99.8%	1.93%	96.2%
MAE Alimentação	Acc TPA <10%	R ² Global	
1.18%	99.2%	0.963	

[!TIP]

Interpretação:

O modelo prevê o ajuste de fotoperíodo com erro médio de **11 minutos** (0.20h). 99.8% das previsões estão dentro de 1 hora do valor ideal. $R^2 = 0.963$ indica que o modelo explica 96.3% da variância dos dados.

7.3 Detalhes do Treino

 Curva de treino do modelo:

Métrica	Valor
Épocas treinadas	139 (early stopping)
Tempo de treino	76,3 segundos
Melhor Val Loss	0,0005
Test MSE	0.0006

7.4 Comparação do modelo neural vs baseline (regras)

 Comparação feita através do notebook:

O modelo mantém boa performance mesmo em condições críticas, com accuracy superior a 96% para erros <1h em todas as faixas.

Conclusões

Objectivos Alcançados

- ☒ Rede neural multi-output funcional com 3 entradas e 3 saídas
- ☒ MAE < 0.2h para fotoperíodo, < 2% para TPA e alimentação
- ☒ $R^2 > 0.96$ indicando excelente capacidade preditiva
- ☒ API REST integrada com o dashboard AquaSense
- ☒ Documentação completa e código modular

Limitações

- Dados sintéticos (não validados com aquários reais)
- Apenas 3 sensores (poderia incluir NO₃, PO₄, O₂)
- Regras base são genéricas (não personalizadas por tipo de aquário)

Trabalho Futuro

- Validação com dados de aquários reais
- Adição de mais sensores
- Sistema de feedback do utilizador para refinamento contínuo
- Diferentes perfis para água doce, salgada, plantado

Conclusão Final: O sistema PhotoperiodNet demonstra que redes neurais podem efectivamente aprender relações complexas entre parâmetros de aquário e fornecer sugestões úteis para manutenção, mesmo

quando treinadas apenas com dados sintéticos baseados em regras especializadas.